

AI ASSISTED CODING

ASSIGNMENT-20.1

HTNO: 2303A510B2

NAME: K AKHIL

BATCH: 14

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Input Validation Check

Task:

Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

PROMPT: #Generate a simple Python program for username and password login.

CODE AND OUTPUT:

```
lab(20.1) > t1.py > ...
1  #Generate a simple Python program for username and password login.
2  username = input("Enter username: ")
3  password = input("Enter password: ")
4  if username == "admin" and password == "admin123":
5      print("Login successful!")
6  else:
7      print("Invalid credentials!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Microsoft VS Code/OneDrive/Documents/AI(vs)/lab(20.1)/t1.py
● Enter username: Akhil
  Enter password: 1203
  Invalid credentials!
○ PS C:\Users\akhil\OneDrive\Documents\AI(vs)> |
```

IMPROVED PROMPT:

#Generate a secure Python login program with input validation.

Use regex for username validation, enforce password rules,
and store password using hashing .

CODE :

```

#Generate a secure Python login program with input validation.Use regex for username validation, enforce password rules,
#and store password using hashing.
import re
import hashlib

# Stored credentials (hashed)
stored_username = "admin"
stored_password_hash = hashlib.sha256("admin123".encode()).hexdigest()

# User input
username = input("Enter username: ").strip()
password = input("Enter password: ").strip()

# Username validation (3-15 chars, letters, numbers, underscore)
if not re.match(r"^[a-zA-Z0-9_]{3,15}$", username):
    print("Invalid username format!")
    exit()

# Password validation (min 8 chars)
if len(password) < 8:
    print("Password must be at least 8 characters long!")
    exit()

# Hash input password
password_hash = hashlib.sha256(password.encode()).hexdigest()

# Authentication
if username == stored_username and password_hash == stored_password_hash:
    print("Login successful!")
else:
    print("Invalid credentials!")

```

OUTPUT:

```

Enter username: admin
Enter password: admin123
Login successful!
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>

```

JUSTIFICATION:

- Input validation ensures only safe and expected data is accepted, reducing security risks.
- Using regex helps prevent malicious inputs such as special characters and injection attempts.
- Password hashing (SHA-256) protects sensitive data from being exposed in plain text.
- Sanitization methods like `.strip()` improve reliability by removing unwanted spaces.
- Overall, secure coding practices enhance the safety, robustness, and trustworthiness of AI-generated code.

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

PROMPT:

Generate a Python script using SQLite to fetch user details based on username.

CODE:

```
lab(20.1) > t2.py >...
1  #Generate a Python script using SQLite to fetch user details based on username.
2  import sqlite3
3  # Connect to SQLite database (or create it if it doesn't exist)
4  conn = sqlite3.connect('users.db')
5  cursor = conn.cursor()
6  # Create a table for users (if it doesn't exist)
7  cursor.execute('''
8  CREATE TABLE IF NOT EXISTS users (
9      id INTEGER PRIMARY KEY,
10     username TEXT NOT NULL UNIQUE,
11     password TEXT NOT NULL
12 )
13 ''')
14 # Insert a sample user (for testing purposes)
15 cursor.execute('''
16 INSERT OR IGNORE INTO users (username, password) VALUES (?, ?)''', ('admin', 'admin123'))
17 conn.commit()
18 # Fetch user details based on username
19 input_username = input("Enter username to fetch details: ").strip()
20 cursor.execute('SELECT * FROM users WHERE username = ?', (input_username,))
21 user = cursor.fetchone()
22 if user:
23     print(f"User ID: {user[0]}, Username: {user[1]}, Password: {user[2]}")
24 else:
25     print("User not found!")
```

OUTPUT:

```
Enter username to fetch details: akhil
User not found!
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

IMPROVED PROMPT:

Generate a secure Python SQLite script using parameterized queries to prevent SQL injection.

CODE:

```
3 import sqlite3
4
5 conn = sqlite3.connect("users.db")
6 cursor = conn.cursor()
7
8 username = input("Enter username: ").strip()
9
10 # Parameterized query (SAFE)
11 query = "SELECT * FROM users WHERE username = ?"
12 cursor.execute(query, (username,))
13
14 result = cursor.fetchall()
15
16 if result:
17     for row in result:
18         print(row)
19 else:
20     print("No user found.")
21
22 conn.close()
```

OUTPUT:

```
Enter username: admin
(1, 'admin', 'admin123')
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

JUSTIFICATION:

1. SQL injection occurs when user input is directly embedded into queries.
2. String concatenation allows attackers to manipulate database queries.
3. Parameterized queries separate SQL logic from user input.
4. This ensures inputs are treated strictly as data, not executable code.
5. Thus, secure queries prevent unauthorized access and protect database integrity.

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

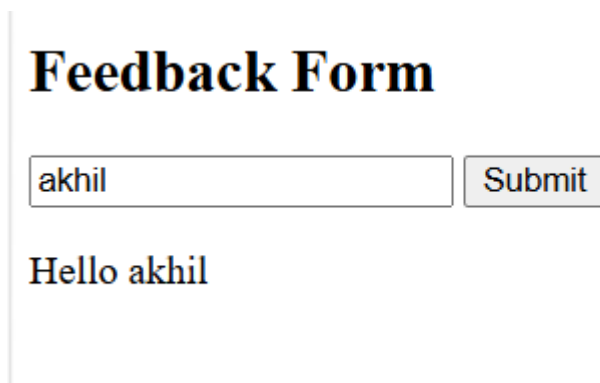
PROMPT:

Generate a simple HTML feedback form with JavaScript that displays user input on the page.

CODE:

```
lab(20.1) > t3.html > html > body
1  #Generate a simple HTML feedback form with JavaScript that displays user input on the page.
2  <!-- Insecure Feedback Form -->
3
4  <!DOCTYPE html>
5  <html>
6  <head>
7  |   <title>Feedback Form</title>
8  </head>
9  <body>
10 |
11 <h2>Feedback Form</h2>
12
13 <input type="text" id="name" placeholder="Enter your name">
14 <button onclick="showFeedback()">Submit</button>
15
16 <p id="output"></p>
17
18 <script>
19 function showFeedback() {
20     var name = document.getElementById("name").value;
21     document.getElementById("output").innerHTML = "Hello " + name;
22 }
23 </script>
24
25 </body>
26 </html>
```

OUTPUT:



Feedback Form

Hello akhil

JUSTIFICATION:

- The original code uses `innerHTML`, which allows execution of malicious scripts.
- This makes the application vulnerable to Cross-Site Scripting (XSS) attacks.
- Untrusted user input must never be directly rendered as HTML.
- Using `textContent` ensures input is treated as plain text, preventing script execution.
- Thus, proper output handling and security practices protect users from XSS vulnerabilities.

Task 4 – Real-Time Application: Security Audit of AI-

Generated Code

Scenario:

Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

Expected Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

PROMPT:

#Generate a simple Python login system using SQLite database.

CODE:

```
lab(20.1) > t4.py > ...
1  #Generate a simple Python login system using SQLite database.
2  # Insecure Login System
3
4  import sqlite3
5
6  conn = sqlite3.connect("users.db")
7  cursor = conn.cursor()
8
9  username = input("Enter username: ")
10 password = input("Enter password: ")
11
12 query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
13 cursor.execute(query)
14
15 if cursor.fetchone():
16     print("Login successful")
17 else:
18     print("Invalid login")
19
20 conn.close()
```

OUTPUT:

```
Enter username: akhil
Enter password: akhil124
Invalid login
```

IMPROVED CODE:

```
lab(20.1) > login.py > ...
1  import sqlite3
2  import hashlib
3  import re
4
5  # Connect to database
6  conn = sqlite3.connect("users.db")
7  cursor = conn.cursor()
8
9  # Take input
10 username = input("Enter username: ").strip()
11 password = input("Enter password: ").strip()
12
13 # Username validation
14 if not re.match(r"^[a-zA-Z0-9_]{3,15}$", username):
15     print("Invalid username format")
16     exit()
17
18 # Password validation
19 if len(password) < 8:
20     print("Password must be at least 8 characters")
21     exit()
22
23 # Hash password
24 password_hash = hashlib.sha256(password.encode()).hexdigest()
25
26 # Secure query (prevents SQL Injection)
27 cursor.execute("SELECT * FROM users WHERE username=? AND password=?",
28               (username, password_hash))
29
30 # Check result
31 if cursor.fetchone():
32     print("Login successful")
33 else:
34     print("Invalid login")
35
36 conn.close()
```

OUTPUT:

```
7 oneb11vc/documents/72(75)/1ab(2072)/10g-mp)
Enter username: admin
Enter username: admin
Enter password: admin123
o Login successful
```

JUSTIFICATION:

1. The insecure code is vulnerable to SQL injection due to string concatenation.
2. Plain text password storage exposes sensitive user data.
3. Lack of input validation allows malicious inputs to enter the system.
4. Parameterized queries and hashing significantly improve security.
5. Secure coding practices ensure safe authentication and protect system integrity.

Task 5 – Insecure Data Serialization Check

Task:

Evaluate AI-generated code that uses pickle for saving user data.

Instructions:

- Ask AI to generate a script using pickle.load().
- Explain why untrusted pickle data is dangerous.
- Replace with safer formats like JSON.

Expected Output:

- Secure serialization code using JSON instead of pickle.

PROMPT:

#Generate a Python script that saves and loads user data using pickle.

CODE:

```

lab(20.1) > t5.py > ...
1 #Generate a Python script that saves and loads user data using pickle.
2 # Insecure serialization using pickle
3
4 import pickle
5
6 # Save data
7 user_data = {"username": "admin", "password": "admin123"}
8
9 with open("data.pkl", "wb") as file:
10 |     pickle.dump(user_data, file)
11
12 # Load data
13 with open("data.pkl", "rb") as file:
14 |     data = pickle.load(file)
15
16 print("Loaded Data:", data)

```

OUTPUT:

```

C:\OneDrive\Documents\AI(vs)\lab(20.1)\t5.py
Loaded Data: {'username': 'admin', 'password': 'admin123'}
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>

```

IMPROVED CODE:

```

lab(20.1) > t5.py > ...
1 #Generate a secure Python script to store and load user data using JSON instead of pickle.
2 # Secure serialization using JSON
3
4 import json
5
6 # Save data
7 user_data = {
8 |     "username": "admin",
9 |     "password": "admin123"
10 | }
11
12 with open("data.json", "w") as file:
13 |     json.dump(user_data, file)
14
15 # Load data
16 with open("data.json", "r") as file:
17 |     data = json.load(file)
18
19 print("Loaded Data:", data)

```

JUSTIFICATION:

1. `pickle.load()` is unsafe because it can execute arbitrary code.
2. Loading untrusted pickle files may lead to serious security attacks.
3. JSON is safer as it stores only data, not executable instructions.
4. It prevents remote code execution vulnerabilities.
5. Therefore, JSON is preferred for secure data serialization.